The `DispatcherServlet` component provides a central entry point for request processing. When a client sends a new HTTP request for a specific URL pattern, `DispatcherServlet` asks the `HandlerMapping` component for the controller responsible for that endpoint, and it finally delegates the actual processing of the request to the specified controller. The controller processes the request, possibly by calling some other services, and then returns a response to `DispatcherServlet`, which finally replies to the client with an HTTP response.

Notice how the Tomcat server is embedded in the Spring Boot application. Spring MVC relies on the web server to accomplish its functionality. The same is true for any web server implementing the Servlet API, but since we are explicitly using Tomcat, let's go ahead and explore some options for configuring it.

3.2.3 Configuring the embedded Tomcat

Tomcat is the default server that comes preconfigured with any Spring Boot web application. Sometimes the default configuration might be enough, but for applications in production, you'll likely need to customize its behavior to meet the specific requirements.

NOTE In a traditional Spring application, you would configure a server like Tomcat in dedicated files such as server.xml and context.xml. With Spring Boot, you can configure an embedded web server in two ways: through properties or in a `WebServerFactoryCustomizer` bean.

This section will show you how to configure Tomcat through properties. You'll learn more about configuring applications in the next chapter. For now, it's enough to know that you can define properties in an application.properties or application.yml file located in the